Date

LECTURE-15

Wednesday 14/10/25

Longest Common Subsequence:

Array = { a, b, d, a, k, a, k, b } length = n

X = { a, b, a, b, d } length = m

Sub-String = { a, b, d, a } ✓ { b, d, a, b } ✗

{ a, k, a, a } ✓ { a, b, k, k } ✗

Sub-Sequence = { a, b, k, b } ✓ { a, a, a, a } ✓ { } ✓

* Sub string needs to be continuous but sub sequence could be all. random combinations from the array

* No. of possible sub sequence in array of size N = 2^N (powerset)

⇒ Brute Force Code:

(1) Generate all subsequences of X of length 'm' (or all seqs. of array with length 'n', depending which is smaller).

(2) Search the subsequences of Array in X (or opposite)

Time Complex = $(2^m) \cdot (n)$

⇒ Recursive Sol:

Date

=) Recursive Solution:

Base case: { x[0] y[m] ; x[n] y[0] }

if n==0 or m==0 length = 0

X = a,b,c
Y = { }

X = { }
Y = a,b,c

When we have either of string empty, that will be base case & that means length of longest common sub seq = 0 as length of empty = 0.

General case:

Last value of both array / string are same

if x[n] == y[m] → X = {a,b,c} , Y = {x,y,c}
1 + LCS(n-1, m-1)

else x[n] ≠ y[m] → Last value of both strings different
max [LCS(n-1, m) , LCS(n, m-1)]

2D Array used for LCS:

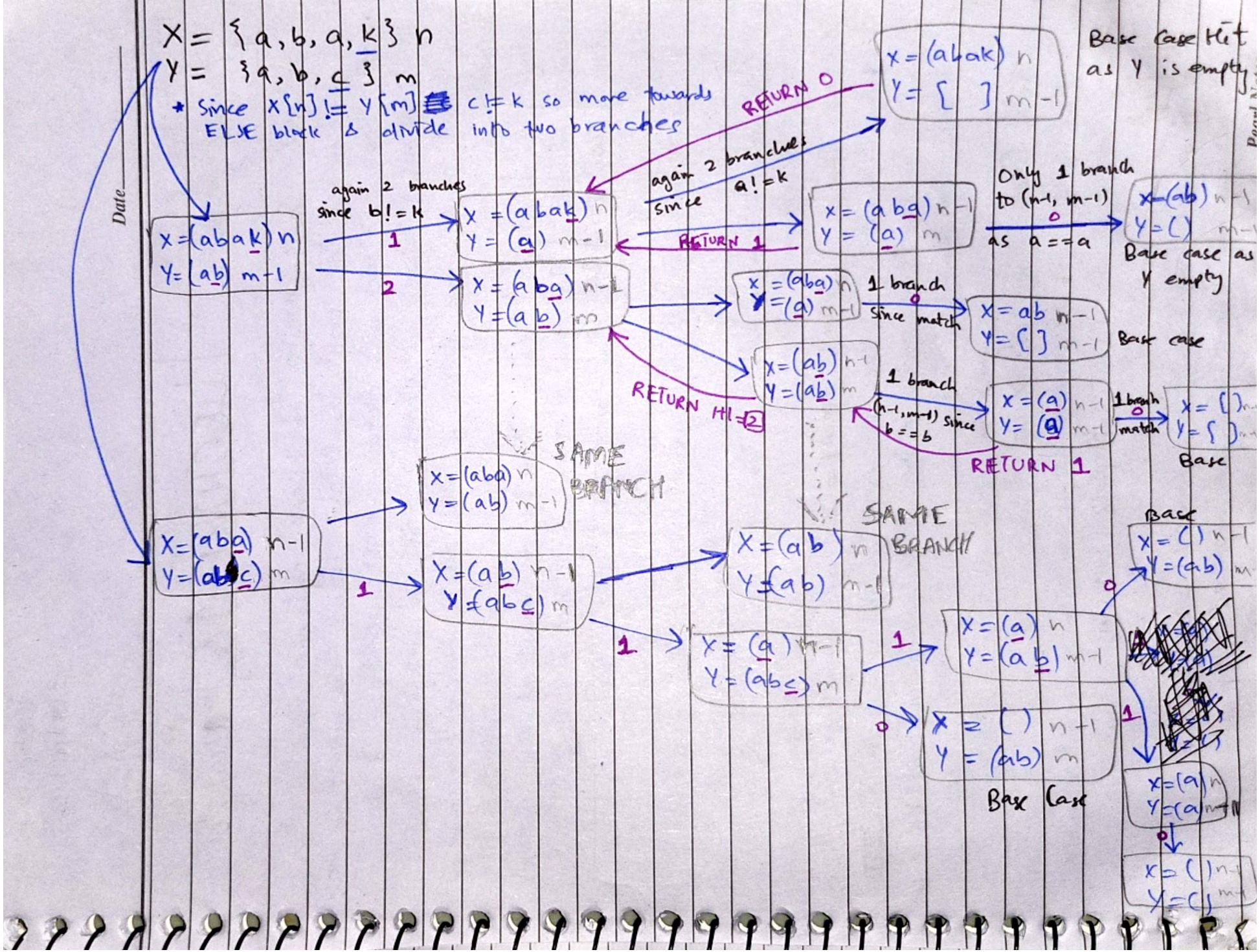
n = 3, m = 3

n ↓ / m →	0	1	2	3
0	0			
1		(1,1)	(1,2)	
2		(2,1)	(2,2)	(2,3)
3			(3,2)	(3,3)

→ (n+1) x (m+1)

Suppose you're standing at (3,3) and x[3] == y[3] ; you'll persue the IF block. In the IF block; we'll move to (2,2) as n-1 & m-1 is (3-1, 2-1). If at (3,3) x[3] ≠ y[3] ; then move in ELSE block & then we'll see the max value out of (2,3) and (3,2) which is (n-1, m) & (n, m-1).

CamScanner


X = {a, b, a, k} n
Y = {a, b, c} m
* Since X[n] != Y[m] i.e. c != k so move towards ELSE block & divide into two branches
x = (abak) n
y = [ ] m-1
Base case Hit as Y is empty
RETURN 0
again 2 branches since b != k
again 2 branches since a != k
x = (abak) n
y = (ab) m-1
x = (aba) n-1
y = (ab) m
RETURN 1
x = (aba) n-1
y = (a) m
Only 1 branch to (n-1, m-1) as a == a
x = (ab) n-1
y = ( ) m-1
Base case as Y empty
x = (aba) n
y = (a) m-1
1 branch since match
x = ab n-1
y = [ ] m-1
Base case
x = (ab) n-1
y = (ab) m
RETURN Hi = 2
1 branch (n-1, m-1) since b == b
x = (a) n-1
y = (a) m-1
RETURN 1
1 branch match
x = ( ) n-1
y = ( ) m-1
Base
SAME BRANCH
x = (aba) n
y = (ab) m-1
SAME BRANCH
x = (aba) n-1
y = (abc) m
x = (ab) n-1
y = (abc) m
x = (ab) n
y = (ab) m-1
x = (a) n-1
y = (abc) m
x = (a) n
y = (ab) m-1
x = ( ) n-1
y = (ab) m
Base Case
Base
x = ( ) n-1
y = (ab) m
x = (a) n
y = (a) m-1
x = ( ) n-1
y = ( ) m-1